

Vantage — Formula Reference

Autonomous Transaction Intelligence for Monad

Every score, threshold, and decision rule in the pre-sign guard pipeline. All formulas are deterministic and evidence-backed; AI is used only for the plain-English explanation.

1. Output Drift (psg/forecast.ts)

Signed percentage difference between what the chain currently returns (simulated) and the quote the wallet showed the user. Measured against the ORIGINAL quote on every recheck so a pool move is never re-baselined away.

```
drift = (simulated - quoted) / quoted × 100
```

Flag:drift < -5 → STALE_STATE

2. Contention Score (psg/forecast.ts)

Call density over a rolling ~3-minute window of event logs on the target contract. Time-based, not block-based, so it captures real wall-clock activity regardless of block speed. Chunked into ≤100-block eth_getLogs queries (Monad RPC cap).

```
contention = min(logs / 20, 1.0)
```

Window:180 s (CONTENTION_WINDOW_SECONDS); block time estimated by sampling latest block twice

Flag:contention ≥ 0.3 → HIGH_CONTENTION

3. Execution Conflict Analyzer (ECA v3) (psg/forecast.ts)

Enumerates Monad's pending block (the only mempool primitive the public RPC exposes) and counts competitors against the evaluated tx. Filters: same-sender txs (nonce-serialized → never competitors) and non-viable txs (explicit gasPrice=0 / gas=0) are excluded. Falls back to log-density × 0.5 when the mempool is unavailable.

Base curve — diminishing returns, monotonic, never fully saturates:

```
g(n, k) = n / (n + k)
```

Sub-scores:

```
sameContractScore = g(competingTxCount, 2)
```

```
directConflictScore = g(effectiveDirect, 1) (AMM swaps)
```

```
directConflictScore = g(competingClaimCount, 0.5) (same-slot claims)
```

```
poolPressureScore = g(viablePendingPoolSize, 250)
```

effectiveDirect = competingSwapCount + 0.5 × competingPoolWriterCount (non-swap reserve movers count half)

Weighted combination, clamped to 0-1:

```
conflictScore = min(1, 0.08·sameContract + 0.86·direct + 0.06·pool)
```

Severity ladder (HIGH subsumes POTENTIAL):

POTENTIAL_STATE_CONFLICTconflictScore ≥ 0.30

HIGH_STATE_CONFLICTconflictScore ≥ 0.60

Design constraints: two competing swaps must score ≥ 0.60 (HOLD anchor), one must stay below it — this pins the direct curve at k=1 and combined non-pool weight ≈0.94. Ambient same-contract + pool pressure alone can never flag (max 0.08 + 0.06 = 0.14).

4. Pre-Submit Validity (psg/forecast.ts)

Three-way nonce check against the chain's pending nonce:

`tx.nonce < pendingNonce` → NONCE_STALE (already used)
`tx.nonce === pendingNonce` → ready to submit
`tx.nonce > pendingNonce` → NONCE_GAP (prior nonces must land first)

Balance check:

`balanceSufficient = balance ≥ gasEstimate × gasPrice + tx.value`

Fallback gas budget: 100,000 gas when estimateGas fails

5. Risk Derivation (psg/forecast.ts)

Maps the flag set to a severity level. Order matters:

- REVERT → CRITICAL
- STALE_STATE | LOW_BALANCE | NONCE_STALE | NONCE_GAP | HIGH_STATE_CONFLICT → HIGH
- HIGH_CONTENTION → HIGH if contention ≥ holdThreshold (calibrated, default 0.7), else MEDIUM
- APPROVAL_EXPOSURE | POTENTIAL_STATE_CONFLICT → MEDIUM
- otherwise → LOW

6. Policy Decision Tree (apa/policy.ts)

- LOW + no flags → PROCEED
- MEDIUM → WARN
- HIGH: NONCE_GAP | NONCE_STALE | LOW_BALANCE → ABORT (absolute, takes precedence over contention)
- HIGH: HIGH_CONTENTION → HOLD_AND_RECHECK
- HIGH: HIGH_STATE_CONFLICT → HOLD_AND_RECHECK (same shape as contention)
- HIGH: STALE_STATE → WARN (waiting cannot fix a stale quote)
- HIGH (unhandled) → WARN
- CRITICAL → ABORT (reason = revertReason)
- Watchdog bias: active critical alert + PROCEED → WARN

Hold & recheck loop: poll getPSGForecast every 3 s against fresh chain state; resolve away from HOLD, or 30 s timeout → SUGGEST_ADJUSTMENT ("increase gas or wait for activity to settle"). The original quote is kept so drift stays real.

7. Contention Threshold Calibrator (em/calibrator.ts)

Adapts each contract's HOLD threshold from real outcome history (last 60 min). Failures = outcome FAILED or policy ABORT.

`failureRate > 0.40` → threshold × 0.9 (more sensitive)
`failureRate < 0.10 (and ≥ 20 samples)` → threshold × 1.05 (relax)

```
next = clamp(threshold × factor, 0.30, 0.95)
```

Watchdog applies a 5-minute cooldown between recalibrations so the poll loop cannot cause exponential decay.

8. Contract Trust Score (services/scoring.ts)

```
score = 100 - 40·failureRate - min(30, alertCount) - 15·contention -  
15·(sampleSize < 5)
```

failureRate = FAILED outcomes or ABORT policies in the last 24 h. alertCount = weighted active alerts: critical 15 / warning 5 / info 1. contention = highest recent persisted contention. Clamped to [0, 100], rounded to 1 decimal.

Zero evaluations → "not yet scored" (score null) — never a numeric 100. An unscored contract is not a proven-safe one.

9. Watchdog Alerts (services/watchdog.ts)

Background 15 s poll over watched contracts. Reserve drift uses the same signed-percentage drift formula between consecutive polls:

```
reserveDrift = (curr - prev) / prev × 100
```

- reserve_drift → |drift| > 5% (critical ≥ 10%)
- contention_spike → contention ≥ 3× prior poll AND ≥ 0.3 floor
- failure_surge → recent failure rate > 25% (60 min window)
- approval_exposure → allowance ≥ 50% of observer balance (also maxUint256 = unlimited)
- inactivity → no logs in 10 min

Alerts auto-resolve when a poll positively observes the condition absent; stale alerts expire after 1 h.

10. Transaction Watcher (apa/policy.ts)

2 s poll, 60 s deadline. Core principle: "not-found ≠ dropped" — an unreachable node (transport error) is never treated as evidence of eviction. A tx is DROPPED only if it was previously seen pending and vanished, or never seen AND past the grace window.

- CONFIRMED → receipt.status = success
- FAILED → receipt reverted; reason re-simulated via eth_call pinned to the exact block
- DROPPED → unseen > 10 s grace (NEVER_SEEN_GRACE_MS) or previously known then gone
- STUCK → still pending after 45 s (STUCK_AFTER_MS)
- NULL_PENDING → pending/propagating, or the chain was unreachable

Flag → Action Index

- REVERT → ABORT · STALE_STATE → WARN · HIGH_CONTENTION → HOLD (if ≥ threshold)
- HIGH_STATE_CONFLICT → HOLD · POTENTIAL_STATE_CONFLICT → WARN
- NONCE_STALE / NONCE_GAP / LOW_BALANCE → ABORT · APPROVAL_EXPOSURE → WARN